



FORMULARIO DE PRODUCCIÓN DE CONTENIDOS

NOMBRE DE LA ASIGNATURA: Desarrollo en plataformas
NOMBRE COMPLETO DEL DOCENTE CONTENIDISTA: Damián Aníbal Nicolalde Rodríguez

DESARROLLO

1. INTRODUCCIÓN DE LA CLASE

Para la Clase 2 daremos un paso natural desde lo que construimos en la Semana 1: pasamos de una página base y accesible a entender cómo se organiza y se comunica un documento web cuando incluye formularios reales. Empezaremos por ver qué nos ofrece HTML5 para “hablar con el usuario” sin complicarnos: tipos de campo que ya validan lo básico, etiquetas que dan nombre accesible, ayudas y errores que se leen con lectores de pantalla, y una estructura semántica que convierte a la página en un mapa claro (landmarks, encabezados y piezas como figure/figcaption o time). La idea es que puedas mirar tu HTML y reconocer de inmediato dónde estás y qué es cada cosa, antes de pensar en estilos o JavaScript.

Luego conectaremos esa base con validación responsable y buenas prácticas WCAG. Verás cuándo dejar que el navegador haga su trabajo (nativo), cuándo sumar reglas de negocio con un poco de JS sin romper la accesibilidad (mensajes anunciados, foco al primer error), y por qué siempre debemos confirmar en el servidor. Cerraremos con el marco POUR (Perceptible, Operable, Comprensible, Robusto) para que tus formularios sean usables con teclado, legibles con zoom y claros para todos. Recuerda: hoy nos concentramos en el por qué y el cómo conceptual; en la práctica repetiremos estos patrones paso a paso para que queden listos para los retos.

Clase 2: HTML5 avanzado y formularios accesibles

RDA 1: Programar aplicaciones web dinámicas, interactivas y con diseño visualmente adaptable, que permita mejorar la experiencia del usuario.

Reto # 1



2.1. HTML5 avanzado y formularios accesibles (teoría)

Piensa en un formulario como una conversación bien guiada. HTML5 ya trae “buenas preguntas” (tipos de campo), “reglas claras” (atributos de validación) y “pistas” (etiquetas y ayudas). Nuestro trabajo es: 1) usar primero lo nativo, 2) nombrar todo con **label**, 3) enlazar ayudas/errores con **aria-describedby**, y 4) dejar que el navegador valide lo básico.

2.1.1. Mapa mental rápido

1. **Una tarea por formulario** (registro, solicitud, pago).
2. **Orden lógico**: obligatorios primero, opcionales después.
3. **Nombre accesible** con label for.
4. **Ayudas/errores** junto al campo y **vinculados**.
5. **Botón principal** único (type="submit").

2.1.2. Tipos de entrada: deja que el navegador ayude

Los tipos de `<input>` activan teclados móviles adecuados y validaciones de formato. Úsalos siempre que exista uno para tu dato.

Tabla 1. *Tipos de <input> útiles y su aplicación práctica*

Tipo	¿Para qué sirve?	Ventaja práctica de UX/A11y
email	Correos	Formato básico + teclado móvil adecuado
url	Enlaces	Valida esquema; reduce errores
tel	Teléfonos	Teclado numérico (móvil)
number	Cantidades exactas	min/max/step; evita letras
range	Preferencias con deslizador	Añade un valor visible con <code><output></code>
date/time	Fechas/horas	Selectores nativos; menos JS
file	Subir archivos	accept, multiple; valida también en servidor
color	Selector de color	Personalización nativa

Nota. Tabla de referencia. Elaboración propia (D. Nicolalde, 2025).

2.1.3. Estructura mínima accesible

```
<form aria-labelledby="t-registro">
  <!-- Título orienta a lectores de pantalla y a cualquier persona -->
  <h2 id="t-registro">Registro</h2>

  <!-- label for => "nombre accesible" del control -->
  <label for="nombre">Nombre completo</label>
  <input id="nombre" name="nombre" type="text"
    required minlength="2" autocomplete="name"
    aria-describedby="hint-nombre">
  <!-- Hint vinculado: se anuncia junto al campo -->
  <small id="hint-nombre">Como figura en tu documento (mín. 2 caracteres).</small>

  <label for="email">Correo institucional</label>
  <input id="email" name="email" type="email"
    required autocomplete="email"
    aria-describedby="err-email">
  <!-- Error: oculto al inicio; se mostrará si falla la validación -->
  <span id="err-email" role="alert" hidden>Usa el formato usuario@dominio.</span>

  <!-- Radios relacionados se agrupan con fieldset+legend -->
  <fieldset>
    <legend>Preferencia de contacto</legend>
    <label><input type="radio" name="pref" value="correo" checked> Correo</label>
    <label><input type="radio" name="pref" value="telefono"> Teléfono</label>
  </fieldset>

  <button type="submit">Crear cuenta</button>
</form>
```

CSS base (estados y foco):

```
/* Visualiza estados nativos sin JS */
input:required:invalid, select:required:invalid { border: 2px solid #c00; }
input:required:valid, select:required:valid { border: 2px solid #0a0; }

/* Foco visible para navegación con teclado */
:focus-visible { outline: 3px solid #000; outline-offset: 2px; }

/* Enlaces para saltar al contenido principal (accesibilidad global) */
.skip-link { position: absolute; left: -999px; top: -999px; }
.skip-link:focus { position: static; background: #eee; padding: .5rem; }
```



2.1.4. Restricciones nativas (sin JS)

Con atributos ya cubres mucha validación:

```
<!-- number con rango y paso: evita valores fuera de rejilla -->
<label for="creditos">Créditos</label>
<input id="creditos" name="creditos" type="number"
  min="1" max="40" step="1" required>

<!-- ID nacional hipotético (10 dígitos): pattern simple y teclado numérico -->
<label for="idnac">Identificación nacional</label>
<input id="idnac" name="idnac" type="text" required
  pattern="^\d{10}$" inputmode="numeric"
  aria-describedby="hint-id">
<small id="hint-id">10 dígitos, sin espacios ni guiones.</small>
```

Si más tarde necesitas controlar todos los mensajes con JS, puedes añadir novalidate en el <form>. De inicio, apóyate en lo nativo.

2.1.5. Mensajes claros y anunciados (accesibilidad)

Un buen mensaje explica la regla y cómo corregir. Si lo generas con JS, marca el control con aria-invalid="true" y anuncia el texto con role="alert".



```
<label for="correo">Correo</label>
<input id="correo" type="email" required aria-describedby="err-correo">
<span id="err-correo" role="alert" hidden>
  Escribe un correo con formato usuario@dominio.
</span>

<script>
  const correo = document.getElementById('correo');
  const err = document.getElementById('err-correo');

  // Validación ligera "en vivo": no bloquea mientras se teclea
  correo.addEventListener('input', () => {
    correo.setCustomValidity(""); // limpia estado previo
    err.hidden = true;           // oculta error por defecto

    if (!correo.checkValidity()) { // usa validación nativa (type="email")
      const msg = 'Escribe un correo con formato usuario@dominio.';
      correo.setCustomValidity(msg); // define texto del error
      correo.setAttribute('aria-invalid','true'); // marca estado
      err.textContent = msg; err.hidden = false; // se anunciará por role="alert"
    } else {
      correo.removeAttribute('aria-invalid');
    }
  });
</script>
```

2.1.6. Autocompletado: accesibilidad también es teclear menos

Usa tokens estándar (given-name, family-name, email, tel, address-line1, postal-code, country) para reducir errores y esfuerzo, sobre todo en móvil y para personas con ayudas técnicas.

```
<label for="nombres">Nombres</label>
<input id="nombres" name="nombres" autocomplete="given-name" required>

<label for="apellidos">Apellidos</label>
<input id="apellidos" name="apellidos" autocomplete="family-name" required>

<label for="telefono">Teléfono</label>
<input id="telefono" name="telefono" type="tel" autocomplete="tel" inputmode="tel">
```

2.1.7. Formularios por pasos

Cuando hay muchos campos, dividir en pasos baja la carga cognitiva. Cada paso debe tener **título** (<h2>), **progreso** (<progress value="1" max="3">) y **gestión del foco** (al avanzar, el foco va al título del nuevo paso). No borres datos al volver.

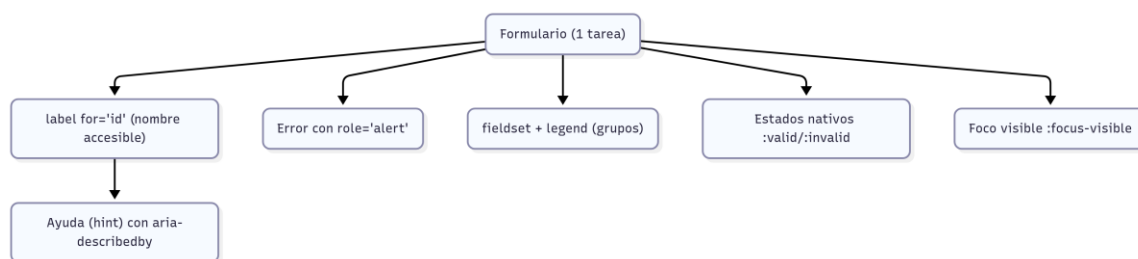


Figura 1. Anatomía de un formulario accesible

Nota. Diagrama conceptual. Elaboración propia (D. Nicolalde, 2025).

2.1.8. ¿Qué valida el navegador y qué agrego yo?

Tabla 2. Estados de validación nativa (ValidityState) y qué comunicar

Flag	Causa típica	Mensaje claro para el usuario
valueMissing	Falta valor (required)	“Completa este campo.”
typeMismatch	email/url mal formateado	“Usa el formato usuario@dominio.”
patternMismatch	No coincide con pattern	“Sigue el formato indicado.”
tooShort/tooLong	Longitud fuera de rango	“Ajusta la cantidad de caracteres.”
rangeUnder/Over	Fuera de min/max	“Elige un valor permitido.”
stepMismatch	No cae en la rejilla de step	“Ajusta al paso (p. ej., múltiplos de 1).”
customError	Error definido por JS	Explica la regla de negocio y solución

Nota. Resumen para guiar mensajes. Elaboración propia (D. Nicolalde, 2025).

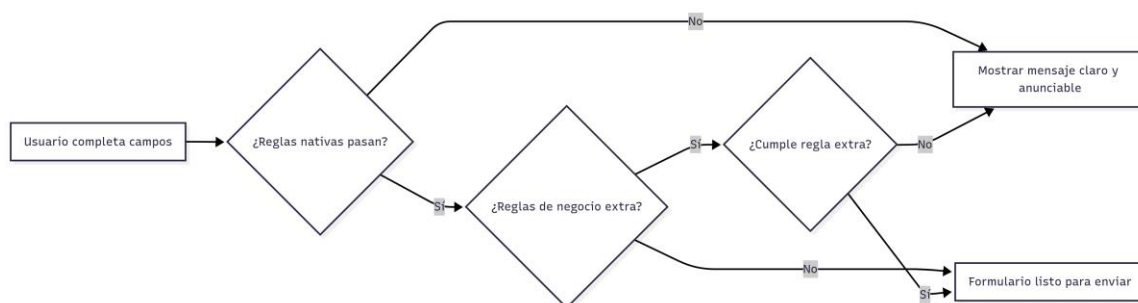


Figura 2. Flujo mental de validación en cliente (nativo + reglas propias)

Nota. Esquema secuencial para decidir qué capa valida y cuándo. Elaboración propia (D. Nicolalde, 2025).

Enlace 1

- **Título del enlace relacionado:** Guía práctica de validación de formularios
- **Descripción:** Introducción clara a validación nativa y personalizada, con ejemplos de `checkValidity()`, `setCustomValidity()` y patrones comunes.



- **Enlace:** https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation

2.1.9. Internacionalización y seguridad (mínimos)

- **Fechas/números:** la UI puede mostrarse local, pero guarda en ISO/UTC.
- **Mensajes:** en el idioma del documento (<html lang="es">), sin tecnicismos.
- **Seguridad:** el cliente mejora la experiencia; el servidor decide. Revalida y sanea, usa **HTTPS**, protege **CSRF** si hay sesión, y verifica **tipo real** y tamaño en archivos (no confíes solo en accept).

2.2. Etiquetas estructurales y semánticas

Piensa que una página responde dos preguntas todo el tiempo: **(a) dónde estoy** (estructura) y **(b) qué es esto** (semántica). Si usamos las etiquetas correctas, la página “habla” por sí misma: los lectores de pantalla pueden saltar por regiones, Google entiende el contenido, y tú (o tu equipo) pueden mantener el código sin adivinar. La regla madre es: **HTML nombra y ordena; CSS acomoda; JS anima.**

2.2.1. Landmarks: las “zonas” navegables

Los *landmarks* son regiones estándar que permiten saltos rápidos (teclado y lectores de pantalla). Los principales:

- header (cabecera global o de sección),
- nav (navegación),
- main (contenido principal, único por documento),
- section (bloque temático con título),
- article (unidad independiente: noticia, ficha, post),
- aside (contenido complementario),
- footer (pie global o de sección).

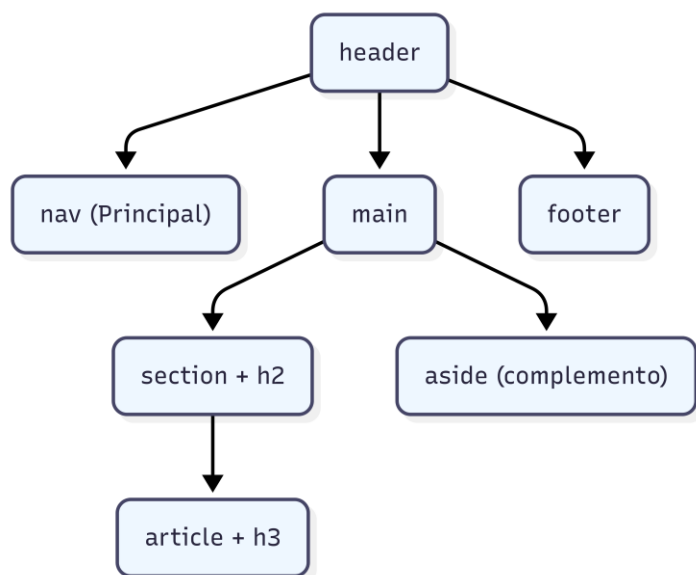


Figura 3. *Mapa de landmarks típicos en una página sencilla*

Nota. Diagrama conceptual de regiones semánticas. Elaboración propia (D. Nicolalde, 2025).



Fragmento base con landmarks:

```
<body>
  <!-- Enlace para saltar directamente al contenido principal (a11y) -->
  <a class="skip-link" href="#contenido">Saltar al contenido</a>

  <header>
    <h1>Plataforma Académica</h1>
    <!-- Menú principal dentro de nav; aria-label distingue si hay varias nav -->
    <nav aria-label="Principal">
      <ul>
        <li><a href="#" aria-current="page">Inicio</a></li>
        <li><a href="#novedades">Novedades</a></li>
        <li><a href="#ayuda">Ayuda</a></li>
      </ul>
    </nav>
  </header>

  <!-- Un único main por documento -->
  <main id="contenido">
    <!-- section: agrupación temática con título propio -->
    <section aria-labelledby="t-nov">
      <h2 id="t-nov">Novedades</h2>

      <!-- article: pieza que tiene sentido por sí sola -->
      <article aria-labelledby="a1">
        <h3 id="a1">Lanzamiento del Panel</h3>
        <p>Publicado el <time datetime="2025-09-22">22/09/2025</time>.</p>
      </article>

      <!-- aside: información complementaria (no esencial) -->
      <aside aria-label="Contactos rápidos">
        Soporte: <a href="mailto:soporte@ejemplo.edu.ec">soporte@ejemplo.edu.ec</a>
      </aside>
    </section>
  </main>

  <footer>© 2025 Plataforma Académica</footer>
</body>
```

2.2.2. Encabezados: el “índice” del documento

Los encabezados (h1–h6) indican jerarquía, no tamaño visual. Reglas prácticas:

1. Un solo h1 (título principal de la página).
2. Secciones con h2; subsecciones con h3, y así sucesivamente.

3. No saltes niveles por estética (el tamaño lo da CSS).
4. Toda section o article debería tener su encabezado.

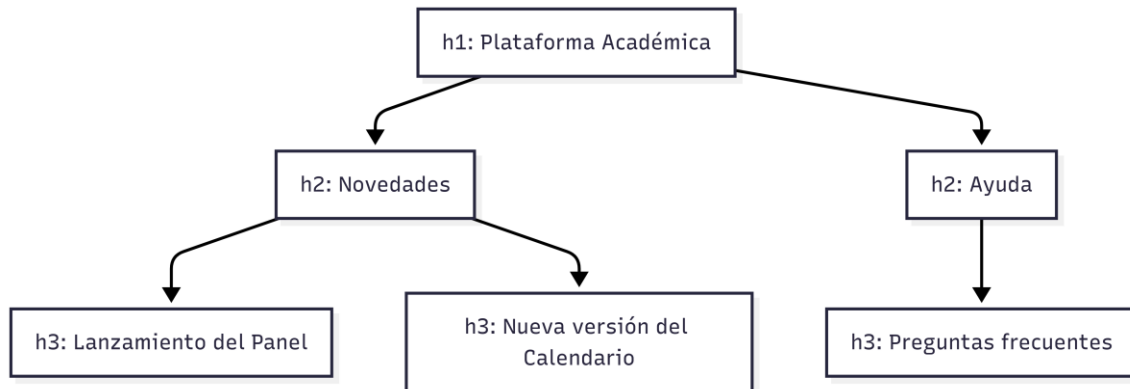


Figura 4. *Jerarquía de encabezados bien formada*

Nota. La jerarquía debe seguir el orden lógico (no saltar de h1 a h4). Elaboración propia (D. Nicolalde, 2025).

2.2.3. Semántica fina: “qué es” cada pieza

Además de las zonas, nombra las piezas internas:

- **Imágenes con contexto:** figure + figcaption. El alt describe la información visual; el pie añade contexto.
- **Fechas:** time datetime="YYYY-MM-DD" (texto humano + valor legible por máquinas).
- **Abreviaturas:** abbr title="..." (p. ej., WCAG).
- **Código/teclado/salida:** code, kbd, samp, y pre para formato preservado.
- **Énfasis/Importancia:** em y strong (significado, no solo estilo).
- **Resaltado contextual:** mark (p. ej., término buscado).
- **Definiciones/Pares clave–valor:** dl + dt + dd.
- **Sección colapsable:** details + summary (acordeón nativo y accesible).



Fragmentos útiles:

```
<!-- Imagen informativa con pie -->
<figure>
  
  <figcaption>Vista del panel tras iniciar sesión (métricas principales).</figcaption>
</figure>

<!-- Fecha legible por humanos + máquina -->
<p>Actualizado el <time datetime="2025-09-22">22 de septiembre de 2025</time>.</p>

<!-- Abreviatura -->
<p>Cumple con <abbr title="Pautas de Accesibilidad al Contenido Web">WCAG</abbr> AA.</p>

<!-- Acordeón nativo para FAQ -->
<details>
  <summary>¿Cómo recupero mi contraseña?</summary>
  <p>Usa la opción "Olvidé mi contraseña" y revisa tu correo institucional.</p>
</details>
```

2.2.4. Listas, definiciones y tablas (bien usadas)

- **Listas:** ul (sin orden), ol (ordenadas). Cada elemento es un li.
- **Definiciones:** dl/dt/dd son excelentes para glosarios o pares clave–valor.
- **Tablas** (solo datos tabulares, no para maquetar):
 - caption (qué estoy viendo),
 - th scope="col|row" para cabeceras,
 - thead, tbody, tfoot si aplica.

Tabla 3. *Mapeo UI → etiqueta semántica recomendada*

Patrón UI	Etiqueta(s) sugerida(s)	Nota de accesibilidad
Cabecera global	header	Puede contener nav/buscador
Menú principal / migas	nav (+ aria-label si hay varias)	Salto rápidos por landmarks
Contenido principal	main (único)	Primer destino tras “Saltar al contenido”
Tarjeta de noticia/ficha	article + h3 + figure/figcaption	Unidad independiente
Fecha de publicación	time datetime="YYYY-MM-DD"	Legible por máquinas
FAQ	details + summary	Nativo y accesible
Panel lateral	aside	Complementario, no esencial
Tabla de datos	table + caption + th scope="..."	No usar para layout

Nota. Tabla de referencia. Elaboración propia (D. Nicolalde, 2025).

Ejemplo de tabla correcta:

```
<table>
  <caption>Estados del sistema por entorno</caption>
  <thead>
    <tr>
      <th scope="col">Entorno</th>
      <th scope="col">Estado</th>
      <th scope="col">Actualizado</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <!-- Cabecera de fila con scope="row" -->
      <th scope="row">Desarrollo</th>
      <td>Operativo</td>
      <td><time datetime="2025-09-22">22/09/2025</time></td>
    </tr>
  </tbody>
</table>
```

2.2.5. ¿section, article o div?

Test rápido:

- ¿Tiene sentido por sí solo (se podría compartir o syndicar)? → article.
- ¿Agrupa contenido relacionado que merece título propio? → section.
- ¿Solo necesitas un contenedor para CSS/JS sin significado? → div.

Tabla 4. Elección entre section, article y div

Pregunta guía	Elemento	Ejemplo típico
¿Vive por sí mismo (unidad independiente)?	article	Noticia, ficha de producto
¿Agrupa temas con un título claro?	section	Bloque “Novedades”, “Ayuda”
¿Solo sirve para estilos/JS?	div	Contenedor de grid/flex sin semántica

Nota. Criterios prácticos de decisión. Elaboración propia (D. Nicolalde, 2025).

2.2.6. ARIA: cuándo sí y cuándo no

ARIA no reemplaza HTML. Primero elige el elemento correcto; luego, si no existe equivalente nativo, usa ARIA para completar:

- Varias nav → diferéncialas con aria-label="Principal", aria-label="Breadcrumbs".



- Acordeón custom (si no usas details): role="button", aria-expanded, aria-controls, y teclas Enter/Espacio.
- Evita roles redundantes: no pongas role="navigation" en nav, ni role="main" en main.

Fragmento (botón colapsable custom, solo como idea):

```
<button aria-expanded="false" aria-controls="bloque1" id="toggle1">
  Ver más
</button>
<div id="bloque1" hidden>
  <p>Contenido adicional...</p>
</div>

<script>
  const btn = document.getElementById('toggle1');
  const panel = document.getElementById('bloque1');
  btn.addEventListener('click', () => {
    const expanded = btn.getAttribute('aria-expanded') === 'true';
    btn.setAttribute('aria-expanded', String(!expanded));
    panel.hidden = expanded; // alterna visibilidad
  });
</script>
```

2.2.7. Anti-patrones comunes (y su arreglo)

- **“Div soup”** (todo con div) → Usa landmarks y semántica fina.
- **Encabezados por tamaño** (usar h4 “porque es más chico”) → Usa el nivel correcto y estiliza con CSS.
- **section sin encabezado** → Añade h2/h3 dentro o usa div.
- **Figuras sin figcaption** cuando la imagen requiere contexto → Añádalo.
- **Tablas para layout** → Maqueta con Flexbox/Grid (Semana 3).
- **Roles ARIA arbitrarios** que contradicen HTML → Valida y elimina lo redundante.

2.3. Validación nativa y personalizada

Cuando validas un formulario, no “castigas” al usuario: lo acompañas. HTML5 ya trae una validación nativa muy útil (tipos, rangos, obligatoriedad) y tú puedes añadir reglas propias con JavaScript sin perder accesibilidad. La idea es simple:

1. **Deja al navegador** lo común (formato de email, rango numérico).
2. **Agrega** con JS lo que es de negocio (ID, fechas relacionadas, disponibilidad).
3. **Revalida en servidor** siempre (seguridad e integridad).

2.3.1. Tres capas que cooperan

Tabla 5. *Capas de validación y responsabilidades principales*



Capa	¿Qué hace mejor?	Ejemplos típicos	Accesibilidad implicada
Nativa (HTML/CSS)	Formatos y restricciones genéricas	type="email", required, min/max, pattern	Estados :invalid/:valid, mensajes del navegador
Personalizada (JS)	Reglas de negocio y cruzadas	"Fin ≥ Inicio", checksum de ID, fuerza de contraseña	aria-invalid, aria-describedby, role="alert"
Servidor	Veracidad y seguridad	Revalidar, normalizar, sanitizar, CSRF, MIME real en file	Mensajes coherentes con la UI; no revelar detalles sensibles

Nota. Tabla de referencia. Elaboración propia (D. Nicolalde, 2025).

2.3.2. Activar la validación nativa (casi sin código)

Empieza por lo más simple: tipos correctos y atributos de restricción. El navegador hace el resto.

```
<form aria-labelledby="t-registro">
  <h2 id="t-registro">Registro</h2>

  <!-- label for = nombre accesible del control -->
  <label for="correo">Correo institucional</label>
  <input id="correo" name="correo" type="email" required
    autocomplete="email" aria-describedby="err-correo">

  <!-- Mensaje de error visible/legible (inicialmente oculto) -->
  <span id="err-correo" role="alert" hidden>Usa el formato usuario@dominio.</span>

  <!-- number con rango y paso -->
  <label for="creditos">Créditos a tomar</label>
  <input id="creditos" name="creditos" type="number"
    min="1" max="40" step="1" required>

  <button type="submit">Crear cuenta</button>
</form>
/* Visualiza estados nativos sin JS */
input:required:invalid { border: 2px solid #c00; } /* inválido */
input:required:valid { border: 2px solid #0a0; } /* válido */

/* Foco visible para teclado */
:focus-visible { outline: 3px solid #000; outline-offset: 2px; }
```

2.3.3. Mensajes claros y anunciados (accesibilidad)

Un buen mensaje dice qué regla se incumple y cómo corregir. Si personalizas mensajes, anúncialos con ARIA.

```
<label for="idnac">Identificación nacional (10 dígitos)</label>
<input id="idnac" name="idnac" type="text" required pattern="^\d{10}$"
  inputmode="numeric" aria-describedby="hint-id err-id">
<small id="hint-id">Escribe 10 dígitos, sin espacios ni guiones.</small>
<span id="err-id" class="error" role="alert" hidden>Revisa el formato (10 dígitos).</span>

<script>
  const idnac = document.getElementById('idnac');
  const err = document.getElementById('err-id');

  // Validación ligera mientras escribe (no bloquea): usa lo nativo
  idnac.addEventListener('input', () => {
    idnac.setCustomValidity("");
    idnac.removeAttribute('aria-invalid');
    err.hidden = true;

    if (!idnac.checkValidity()) {
      const msg = 'Revisa el formato: exactamente 10 dígitos.';
      idnac.setCustomValidity(msg); // mensaje personalizado
      idnac.setAttribute('aria-invalid', 'true');
      err.textContent = msg; err.hidden = false; // se anuncia por role="alert"
    }
  });
</script>
```

2.3.4. ¿Cuándo validar? Ritmo y tiempos

No interrumpas a cada tecla. Usa una estrategia combinada:

- **onblur** (al salir del campo): formato/obligatoriedad.
- **oninput** con *debounce* (200–400 ms): contadores o “fuerza” de contraseña.
- **onsubmit**: última barrera; enfoca el primer error.

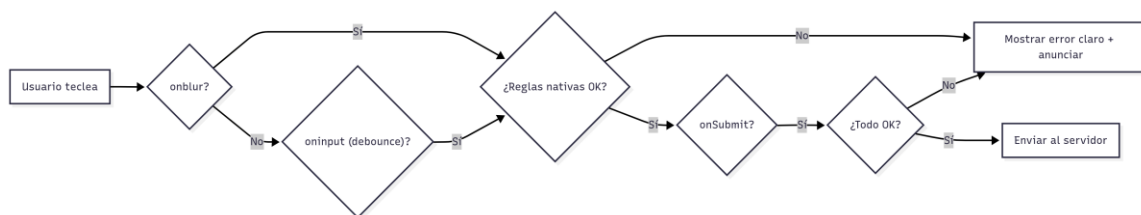


Imagen 5. *Flujo de decisión de validación (cliente)*

Nota. La validación se anuncia sin bloquear la escritura. Elaboración propia (D. Nicolalde, 2025).

2.3.5. Reglas cruzadas: campos que dependen entre sí

Ejemplo clásico: “Fin ≥ Inicio”. Valida cuando cambie cualquiera.



```
<label for="inicio">Fecha inicio</label>
<input id="inicio" type="date" required>

<label for="fin">Fecha fin</label>
<input id="fin" type="date" required aria-describedby="err-fechas">
<span id="err-fechas" role="alert" hidden>La fecha de fin debe ser posterior a inicio.</span>

<script>
  const ini = document.getElementById('inicio');
  const fin = document.getElementById('fin');
  const err = document.getElementById('err-fechas');

  function validarRango() {
    // Limpia estado anterior
    fin.setCustomValidity(''); fin.removeAttribute('aria-invalid'); err.hidden = true;

    // Si ambos valores existen, compara
    if (ini.value && fin.value && fin.value < ini.value) {
      const msg = 'La fecha de fin debe ser posterior a la de inicio.';
      fin.setCustomValidity(msg);
      fin.setAttribute('aria-invalid', 'true');
      err.textContent = msg; err.hidden = false;
    }
  }
  ini.addEventListener('change', validarRango);
  fin.addEventListener('change', validarRango);
</script>
```

2.3.6. Validación asíncrona (servidor): “¿Este correo existe?”

Indica estado, evita “carreras” y no satures al servidor.



```
<label for="correo2">Correo</label>
<input id="correo2" type="email" required aria-describedby="st-correo2 err-correo2">
<!-- role="status" anuncia cambios no urgentes -->
<small id="st-correo2" role="status" aria-live="polite"></small>
<span id="err-correo2" role="alert" hidden></span>

<script>
  const correo2 = document.getElementById('correo2');
  const st = document.getElementById('st-correo2');
  const er = document.getElementById('err-correo2');

  let timer; // para debounce
  let lastQuery = 0; // para evitar respuestas "viejas"

  correo2.addEventListener('input', () => {
    // Limpieza de mensajes y estado
    correo2.setCustomValidity(""); correo2.removeAttribute('aria-invalid');
    er.hidden = true; st.textContent = "";

    // Si el formato ya es inválido, no consultes servidor
    if (!correo2.checkValidity()) return;

    // Debounce: espera 300ms desde la última tecla
    clearTimeout(timer);
    timer = setTimeout(async () => {
      const qid = ++lastQuery; // marca esta consulta
      st.textContent = 'Comprobando disponibilidad...';

      try {
        // Simulación de endpoint (reemplaza por tu API)
        // const resp = await fetch('/api/email/check?e=' + encodeURIComponent(correo2.value));
        // const { exists } = await resp.json();
        const exists = false; // ← simulado

        if (qid !== lastQuery) return; // ignora si llegó tarde
        st.textContent = exists ? 'Este correo ya tiene cuenta.' : 'Disponible.';
        if (exists) {
          const msg = 'Este correo ya está registrado. ¿Quieres iniciar sesión?';
          correo2.setCustomValidity(msg);
          correo2.setAttribute('aria-invalid', 'true');
          er.textContent = msg; er.hidden = false;
        }
      } catch (e) {
        if (qid !== lastQuery) return;
        st.textContent = 'No se pudo comprobar ahora. Intenta luego.';
      }
    }, 300);
  });
```

</script>

2.3.7. Disparadores y ritmo (resumen visual)

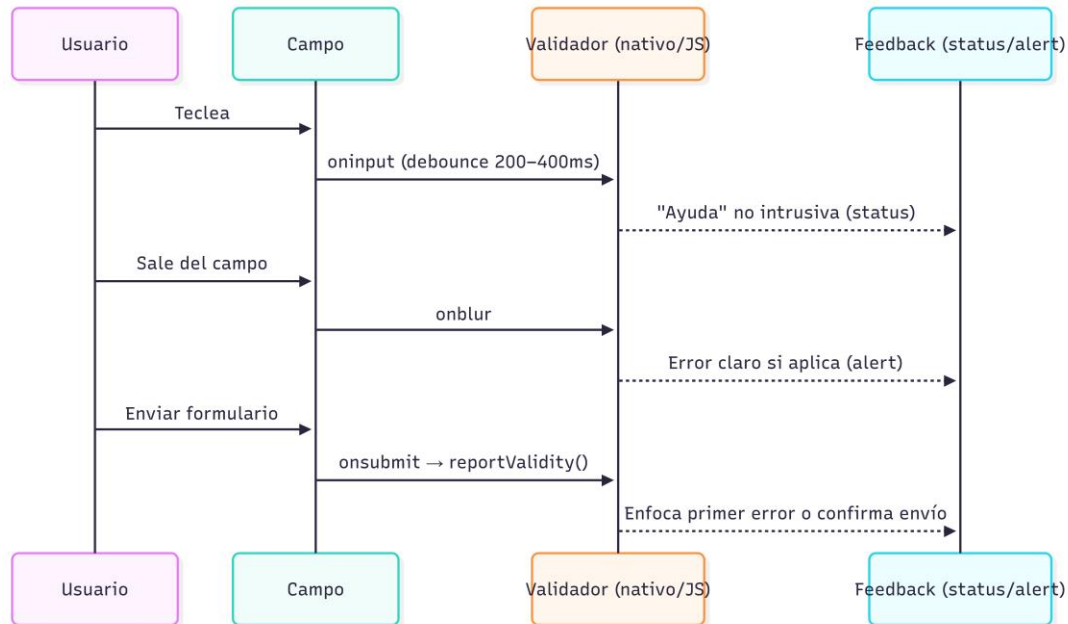


Imagen 6. Línea temporal de disparadores de validación y feedback

Nota. Primero ayudas “suaves”, luego errores “claros”, sin bloquear. Elaboración propia (D. Nicolalde, 2025).

2.3.8. Internacionalización y máscaras

- **Fechas/números:** la UI puede ser local, pero envía/guarda en ISO/UTC.
- **Mensajes:** en el idioma del documento (`lang="es"`), con verbos simples y acción sugerida.
- **Máscaras:** si usas, que no impidan seleccionar/pegar ni rompan lectores; a veces `pattern + hint` es suficiente.

2.3.9. Anti-patrones comunes

- **Placeholders como etiquetas** → Usa `label for`.
- **Solo color para errores** → Añade texto y `role="alert"`.
- **Quitar outline** → Personaliza el foco, no lo elimines.
- **disabled para “solo lectura”** → Usa `readonly`; `disabled` saca del foco y del envío.
- **Bloquear la escritura** con validación “en vivo” agresiva → Usa *debounce* o *onblur*.
- **Modales para cada error** → Mensajes junto al campo + (opcional) resumen arriba con anclas.

2.4. Buenas prácticas de accesibilidad (WCAG)

Cuando hablamos de accesibilidad no estamos “añadiendo algo al final”, estamos diseñando para todas las personas desde el principio. Las WCAG organizan sus pautas en cuatro principios que recordamos como **POUR: Perceptible, Operable, Comprensible y Robusto**. En clase trabajaremos con el objetivo **AA** como mínimo razonable.

2.4.1. Mapa rápido (POUR → acción → prueba)

Tabla 6. *POUR: mapa de acciones concretas y cómo probarlas*

Principio	Acción práctica (AA)	Cómo probar rápido
Perceptible	alt útil en imágenes informativas; figure+figcaption si hace falta contexto	Navega con SR (NVDA/VoiceOver); inspecciona alt
Perceptible	Contraste $\geq 4.5:1$ (texto normal) y $\geq 3:1$ (texto grande)	Lighthouse/Axe; comprobador de contraste
Perceptible	Reflow con zoom 200–400% sin scroll horizontal general	Ctrl/Cmd +; revisa columnas y tarjetas
Operable	Todo con teclado: Tab/Shift+Tab/Enter/Espacio	Recorre la página sin mouse
Operable	Foco visible y orden lógico	:focus-visible; prueba Tab en toda la UI
Operable	Modales accesibles (foco atrapado legit., aria-modal, cerrar con Esc)	Abrir/cerrar con teclado y lector
Comprensible	Etiquetas reales + aria-describedby para ayudas/errores	SR anuncia propósito + error
Comprensible	Jerarquía de encabezados (h1→h2→h3...) y navegación coherente	Vista de encabezados; inspección semántica
Robusto	Semántica nativa primero; ARIA solo si falta equivalente	Validador HTML; roles correctos en DevTools

Nota. Tabla de referencia. Elaboración propia del autor, Damián Nicolalde (2025).

2.4.2. Perceptible: que la información “llegue”

Imágenes. Si la imagen comunica algo, pon alt descriptivo; si es decorativa, alt="". Si requiere contexto, usa figure + figcaption.

```
<!-- Imagen informativa: alt describe la información visual -->
<figure>
  
  <figcaption>Arquitectura lógica del sistema presentada en clase.</figcaption>
</figure>
```

Contraste y reflow. No depende solo del color. Verifica contraste y que con zoom 200–400% la página no se rompa.

```
/* Respetar a usuarios que reducen animaciones por mareo o concentración */
@media (prefers-reduced-motion: reduce){
  * { animation-duration: .01ms !important; animation-iteration-count: 1 !important; }
}
```

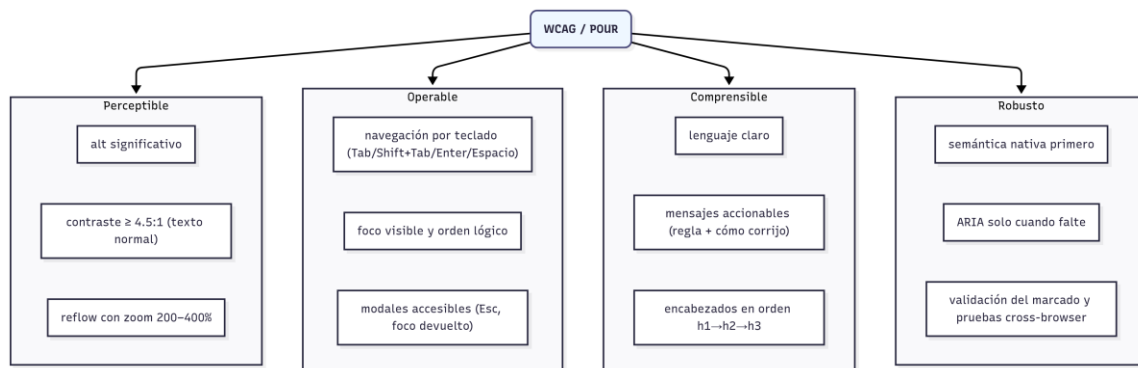


Imagen 7. Esquema mental de los cuatro principios WCAG (POUR)

Nota. Diagrama conceptual. Elaboración propia del autor, Damián Nicolalde (2025).

2.4.3. Operable: teclado first

El sitio debe poder usarse completamente con teclado. Asegura foco visible y orden lógico.



```
<!-- Enlace para saltar al contenido principal: mejora enorme con teclado/SR -->  
<a class="skip-link" href="#contenido">Saltar al contenido</a>
```

```
<!-- ... -->
```

```
<main id="contenido">  
  <!-- contenido principal -->  
</main>
```

```
/* Foco visible: nunca lo elimines; personalízalo si quieres */  
:focus-visible { outline: 3px solid #000; outline-offset: 2px; }
```

```
/* Skip link: oculto visualmente, visible al enfocarse */  
.skip-link { position:absolute; left:-999px; top:-999px; }  
.skip-link:focus { position: static; background:#eee; padding:.5rem; }
```

Modales accesibles (idea mínima). Deben: 1) atrapar foco, 2) anunciarse, 3) cerrarse con Esc, 4) devolver el foco al disparador.

```
<button id="abrir">Abrir modal</button>

<!-- role="dialog" y aria-modal anuncian que es un modal real -->
<div id="modal" role="dialog" aria-modal="true" aria-labelledby="t-modal" hidden>
  <div class="caja">
    <h2 id="t-modal">Confirmar acción</h2>
    <p>¿Deseas continuar?</p>
    <button id="cerrar">Cancelar</button>
    <button id="ok">Sí, continuar</button>
  </div>
</div>

<script>
const abrir = document.getElementById('abrir');
const modal = document.getElementById('modal');
const cerrar = document.getElementById('cerrar');
const ok = document.getElementById('ok');
let ultimoFoco;

function abrirModal(){
  ultimoFoco = document.activeElement;  // guarda quién tenía el foco
  modal.hidden = false;
  modal.querySelector('.caja').focus();  // mueve el foco al modal
  document.addEventListener('keydown', trap);
}

function cerrarModal(){
  modal.hidden = true;
  document.removeEventListener('keydown', trap);
  if (ultimoFoco) ultimoFoco.focus();  // devuelve el foco al disparador
}

function trap(e){
  if (e.key === 'Escape') return cerrarModal();
  // (Aquí iría el ciclo de foco si la caja tuviera muchos elementos)
}

abrir.addEventListener('click', abrirModal);
cerrar.addEventListener('click', cerrarModal);
ok.addEventListener('click', cerrarModal);
</script>
```



Imagen 8. Recorrido de foco recomendado en una página estándar



Nota. Flujo de foco simplificado para navegación por teclado. Elaboración propia del autor, Damián Nicolalde (2025).

2.4.4. Comprensible: que se entienda lo que pides

Usa lenguaje claro y etiquetas reales. Las ayudas y errores deben estar vinculados y anunciarse.

```
<label for="email">Correo institucional</label>
<input id="email" type="email" required aria-describedby="hint-email err-email">
<small id="hint-email">Formato: usuario@dominio.</small>
<span id="err-email" role="alert" hidden>Revisa el formato del correo.</span>
```

Mantén jerarquía de encabezados (h1→h2→h3...) y patrones de navegación consistentes entre páginas (mismo lugar para menú, buscador, etc.).

Tabla 7. Mensajes accionables: patrón “regla + cómo corrijo”

Caso	Evitar	Preferir
Email inválido	“Inválido”	“Usa el formato usuario@dominio.”
Longitud	“Error en texto”	“Mínimo 8 caracteres; añade 2 más.”
Rango	“Número fuera de rango”	“Elige entre 1 y 40.”
Campo requerido	“Campo vacío”	“Completa este campo.”

Nota. Ejemplos de redacción. Elaboración propia del autor, Damián Nicolalde (2025).

2.4.5. Robusto: que funcione “a prueba de futuro”

Semántica nativa primero: header, nav, main, section, article, aside, footer, figure/figcaption, time, dl/dt/dd, details/summary.

ARIA solo cuando falte semántica nativa o para desambiguar (ej.: varias nav con aria-label).

Valida HTML/CSS, prueba en más de un navegador y, si puedes, con un lector (NVDA en Windows, VoiceOver en macOS). Evita roles redundantes (role="navigation" en nav) o incorrectos (pueden confundir a las ayudas técnicas).

Enlace 2

- **Título del enlace relacionado:** WCAG 2.2 – Lista de pautas (visión general)
- **Descripción:** Panorama de las cuatro categorías (POUR) y objetivos AA más relevantes para formularios y estructura semántica.
- **Enlace:** <https://www.w3.org/WAI/standards-guidelines/wcag/>

2.4.6. Anti-patrones frecuentes

- **Quitar el foco por estética** → Personalízalo, no lo elimines.
- **Color como única señal** → Acompáñalo con texto/ícono.



- **Placeholders como etiquetas** → Usa label for.
- **Tablas para maquetar** → Usa Flexbox/Grid (Semana 3).
- **Componentes custom sin roles/teclas** → Usa nativos o implementa roles y teclado completos.
- **Modales que no devuelven el foco** → Devuélvelo al disparador.

Referencias citadas en la Clase 2.

- World Wide Web Consortium. (2018). *HTML 5.2: W3C Recommendation*. W3C.
- World Wide Web Consortium. (2018–2023). *WAI-ARIA Authoring Practices*. W3C.
- World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C.
- Mozilla Developer Network. (s. f.). *Web forms — Working with user data*. MDN Web Docs.
- Mozilla Developer Network. (s. f.). *HTML: Living Standard — Form validation*. MDN Web Docs.
- WHATWG. (s. f.). *HTML Living Standard* (secciones: forms, semantics).

Definición de términos

- **Landmark:** Región semántica de alto nivel (p. ej., header, nav, main, aside, footer) que facilita la navegación por teclado y lectores de pantalla, permitiendo “saltos” rápidos entre zonas de la página. (Término resaltado en azul dentro del texto principal; en Formulario 2 se pide subrayar/resaltar y definir 2 términos).
- **aria-describedby:** Atributo ARIA que **vincula** un control con uno o varios elementos que contienen ayudas y/o errores. Cuando existe, los lectores de pantalla leen **el control + sus ayudas/errores** como un solo bloque informativo, mejorando la comprensión. (Término resaltado; definición al final).